Architectural Refactoring Report: Persistent Mastery Integration and Project Registry Consolidation

Executive Summary

This comprehensive research report details the strategic architectural refactoring of the Stela platform, specifically addressing the integration of "Persistent Mastery" (Skills Crystallization) and the formalization of the "Project Registry" landscape. This initiative is derived from the synthesis of draft intent **DP-OPS-0009** (renumbered herein to **DP-OPS-0010**) and the "Projects Summary Notes" regarding the Living Surface architecture.

The Stela repository currently operates on a strict "Stateless Governance" model.[1] However, this has led to a "Paradox of Memory" where valuable patterns are lost. The user's notes (PROJECTS_SUMMARY_NOTES.md) propose a "Living Surface" architecture to solve this, introducing root-level intake files (SKILL.md, PROJECT.md) that function identically to TASK.md: a stable protocol at the top and an append-only log at the bottom.[1]

This report outlines the execution of **DP-OPS-0010**. It shifts from a purely library-based approach to a "Factory" model:

1. **SKILL.md (Root)**: A living intake surface for capturing candidate patterns before they are crystallized into docs/library/skills/.
2. **PROJECT.md (Root)**: A living lifecycle controller for creating and managing project units.
3. **ops/lib/manifests/MANIFEST.md**: The renamed SSOT for context (formerly CONTEXT.md), adhering to the "No Underscore Soup" rule.
4. **projects/<slug>/STELA.md**: A standardized signature file generated for every project.

---

1. Governance Context and System Constitution

To properly situate the proposed changes, one must first deconstruct the existing governance framework of the Stela platform. The repository is not merely a collection of code but a constitutionally governed environment where "Logic" and "Authority" are strictly delineated.

1.1 The Doctrine of Statelessness

The foundational philosophy of the platform is encoded in **docs/CONTEXT.md**, which asserts that context is the "precise set of artifacts required to rehydrate an agent" and that "if it's not in the Context, it doesn't exist".[1] This doctrine is a defense mechanism against "Drift"—the

gradual entropy that occurs when undocumented assumptions accumulate in a system.

In the current state, the system relies on **Ephemeral Context**. However, the user's research notes introduce the concept of **"Living Surfaces"**—root-level files that bridge the gap between ephemeral work and permanent libraries. Just as TASK.md manages the *current* thread, SKILL.md and PROJECT.md will manage *learning* and *lifecycles*, respectively.[1]

1.2 The "Tone Lanes" Separation

A critical invariant in the repository's structure is the separation of "Tone Lanes" defined in **docs/GOVERNANCE.md**.[1] The new architecture respects this by separating the **Intake** (Root) from the **Archive** (Library):

Surface	Location	Type	Role
Work Surface	TASK.md	Living	Execution & Thread Log
Skill Intake	SKILL.md	Living	Pattern Candidate Log [1]
Project Intake	PROJECT.md	Living	Project Factory & Lifecycle [1]
Skill Library	docs/library/skills/	Static	Crystallized, Audited Protocols [1]
Registry	ops/lib/manifests/	Static	Machine-readable Indices [1]

1.3 The "Manifest Supremacy" and Context Linting

The repository enforces its truth through "Manifest Supremacy." To align with the "No Underscore Soup" recommendation in the notes, the existing ops/lib/manifests/CONTEXT.md will be renamed to **ops/lib/manifests/MANIFEST.md**. This cleaner naming convention signals that this file is *the* Manifest for the platform.[1]

---

2. The Project Landscape Expansion (Registry Refactor)

The user's request to "account for the now bigger project landscape" necessitates a

fundamental shift from implicit discovery to explicit lifecycle management.

2.1 Root Intake: PROJECT.md

The note regarding **PROJECT.md** describes it as the "Project Intake and Project Change Order surface."[1] It is not a registry list; it is a **Factory**.

- **Top Half (Protocol):** Defines how to create, archive, or split a project.
- **Bottom Half (Log):** An append-only audit trail of project lifecycle events (e.g., "Created projects/php-nuke-ce", "Archived projects/old-test").

2.2 The Signature: STELA.md

The notes specify that every project must have a signature file: **projects/<slug>/STELA.md**.[1] This replaces the ambiguous README.md as the machine-readable definition of a project unit. It will contain:

- **Project ID/Slug**
- **Type** (Module, Theme, Core)
- **Status** (Active, Maintenance)
- **Local Context Pointers** (what docs/ops files this project relies on).

2.3 The Registry: ops/lib/manifests/PROJECTS.md

While PROJECT.md tracks *events*, **ops/lib/manifests/PROJECTS.md** tracks *state*. This file will be the Single Source of Truth (SSOT) for tooling like ops/bin/dump.[1] It solves the "Underscore Soup" problem by providing a clean, lintable list of active project paths.

---

3. Persistent Mastery: The Skills Crystallization Subsystem

The integration of "Persistent Mastery" addresses the core weakness of statelessness: the inability to retain complex procedural knowledge. This architecture splits "Learning" into two phases: **Intake** (Candidate) and **Crystallization** (Library).

3.1 Root Intake: SKILL.md

The **SKILL.md** file sits at the repo root. It operates exactly like TASK.md:[1]

- **Protocol (Top):** "Extraction Criteria"—definitions of what constitutes a valid skill (e.g., "Must be generalizable," "Must have invariants").
- **Candidate Log (Bottom):** A raw log where agents append "Skill Candidates" discovered during a session.
    - *Example Entry:* "2026-01-31: Found a pattern for database migration. Draft content:

[link to snippet]."

This ensures that "learning" is a fast, low-friction add operation, rather than a heavy "create new file and PR it" operation in the middle of a task.

3.2 The Library: docs/library/skills/

Once a candidate in SKILL.md is reviewed and approved (Crystallized), it is moved to **docs/library/skills/** as a permanent artifact (e.g., S-OPS-001.md).

- **Template:** The structure of these finalized files is defined in docs/library/skills/TEMPLATE.md (Trigger, Invariants, Protocol, Verification).[1]
- **S-LEARN-01:** This specific skill is the "Meta-Skill" that teaches an agent how to read SKILL.md, extract a candidate, and format it into a library file.[1]

---

4. Operational Logic and Agent Protocol Updates

The introduction of these new subsystems requires updates to the **Behavioral Logic** governing the agents.

4.1 Integration Logic Updates

The **Integrator (Lead AI)** role must be updated to utilize the new Living Surfaces.

- **Intake First:** When a new pattern is found, do not create a library file immediately. Log it to SKILL.md.
- **Factory First:** When a new project is requested, do not mkdir. Log the intent to PROJECT.md.

4.2 Closeout Protocol (TASK.md Implications)

The TASK.md template will be updated to include **SKILL.md** in the "Work Log" check.

- *Current:* "Append a TASK Work Log entry."
- *New:* "Append to TASK.md (Work) and SKILL.md (Learning) if applicable."

---

5. Execution Plan: Drafting DP-OPS-0010

The following section presents the **Dispatch Packet (DP)**. This artifact is the operational output of this research. It is formatted to be copied directly into the TASK.md of the active repository.

DP-OPS-0010 Artifact

DP-OPS-0010: Living Surfaces & Registry Refactor

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: main

Required Work Branch: work/living-surfaces-init

Base HEAD: e29ab6b23

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

Worker must confirm loaded before acting:

- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - ops/lib/manifests/CONTEXT.md
  - docs/library/INDEX.md
  - docs/library/MANUAL.md
  - docs/library/MAP.md

II) SCOPE & SAFETY

- **Objective:** Establish the "Living Surface" architecture (SKILL.md, PROJECT.md) and refactor the manifest layer (CONTEXT.md -> MANIFEST.md, add PROJECTS.md).
- **Non-Goals:**
  - No changes to existing project payloads (content).

- No changes to ops/bin/dump binary logic (only manifest content changes).

Target Files allowlist (hard gate)

- SKILL.md (new)
- PROJECT.md (new)
- docs/library/skills/TEMPLATE.md (new)
- docs/library/skills/S-LEARN-01.md (new)
- docs/library/INDEX.md
- docs/library/MAP.md
- docs/library/MANUAL.md
- ops/lib/manifests/CONTEXT.md (delete/move)
- ops/lib/manifests/MANIFEST.md (new)
- ops/lib/manifests/PROJECTS.md (new)
- TRUTH.md
- SoP.md
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Stop Conditions:**
  - STOP if placeholder text remains.
  - STOP if required DP inputs are missing.

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The system lacks persistent memory for skills and explicit lifecycle management for projects. "Underscore soup" naming (CONTEXT_MANIFEST.md) persists in notes.
- **Drift:** Ad-hoc learning is lost. Project creation is unguarded.
- **Desired State:** Root-level SKILL.md and PROJECT.md exist as intake surfaces. MANIFEST.md and PROJECTS.md exist as strict registries.

B) REQUEST

Numbered tasks (concrete, no vibes):

1. **Establish Root Living Surfaces:**
   - Create **SKILL.md** at repo root. Format: Top section "Protocol: Candidate Extraction", Bottom section "Candidate Log".
   - Create **PROJECT.md** at repo root. Format: Top section "Protocol: Project Factory", Bottom section "Lifecycle Log".
2. **Refactor Manifests:**

    - Rename ops/lib/manifests/CONTEXT.md to **ops/lib/manifests/MANIFEST.md**.
    - Create **ops/lib/manifests/PROJECTS.md**. Content: Empty table/list for registering active projects.
    - Update ops/bin/open and docs/CONTEXT.md to point to the new MANIFEST.md path.
3. **Establish Skills Library:**
    - Create docs/library/skills/.
    - Create **docs/library/skills/TEMPLATE.md** (Schema: Trigger, Invariants, Protocol, Verification).
    - Create **docs/library/skills/S-LEARN-01.md** (The "Crystallization" skill that reads SKILL.md and outputs a library file).
4. **Update Registries & Maps:**
    - Update docs/library/INDEX.md to register the new skills files and the new Root Surfaces (SKILL.md, PROJECT.md).
    - Update docs/library/MAP.md to explain the "Intake vs. Library" flow.
    - Update docs/library/MANUAL.md with commands for logging to SKILL.md.
5. **Define Project Signature:**
    - Create a definition (in PROJECT.md protocol section) that every project must contain a **STELA.md** signature file.
6. **Update Constitution (TRUTH.md):**
    - Update "Section 3 (Structure)" to formally recognize the 3 Root Surfaces: TASK.md (Work), SKILL.md (Learning), PROJECT.md (Factory).
7. **Update State of Play (SoP.md):**
    - Log the architectural refactor.

C) CHANGELOG

- **Log:**
    - Established SKILL.md and PROJECT.md as root living surfaces.
    - Renamed CONTEXT.md to MANIFEST.md and created PROJECTS.md registry.
    - Established docs/library/skills/ subsystem.
    - Defined STELA.md project signature standard.

D) PATCH / DIFF

- **Format:** Unified diff.

E) RECEIPT (REQUIRED)

Verification

- ./ops/bin/dump --scope=platform
- bash tools/context_lint.sh (Must pass; requires updating the script to check MANIFEST.md instead of CONTEXT.md).
- bash tools/lint_library.sh

- bash tools/lint_truth.sh

Artifact bundle

- storage/handoff/DP-OPS-0010-RESULTS.md
- git diff --stat

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties

- [ ] Produce storage/handoff/DP-OPS-0010-RESULTS.md.
- [ ] Update **SoP.md**.
- [ ] Append to **TASK Work Log**.

---

6. Verification Framework and Compliance Analysis

6.1 The Manifest Rename (tools/context_lint.sh)

The renaming of CONTEXT.md to MANIFEST.md will break tools/context_lint.sh.

- **Resolution:** The DP execution must include updating tools/context_lint.sh to look for ops/lib/manifests/MANIFEST.md. This is an implicit requirement of Task 2.

6.2 The Living Surface Pattern

The introduction of SKILL.md and PROJECT.md mimics TASK.md.

- **Verification:** tools/verify_tree.sh (if it exists or is strict) might flag these new root files as "clutter". The DP should ensure TRUTH.md allows them, so future linting passes them.

6.3 Project Signature (STELA.md)

The requirement for STELA.md in projects is a "Day 2" compliance goal.

- **Strategy:** This DP defines the standard in PROJECT.md, but does not enforce retroactive creation of STELA.md files in existing projects (to keep the DP scoped). A follow-up DP (DP-OPS-0011) should populate them.

---

7. Conclusion

This updated architecture aligns with the "Living Surface" philosophy found in the research notes. By establishing **SKILL.md** and **PROJECT.md** as root-level intake protocols, the system gains a high-speed way to capture value (learning and lifecycles) without bogging down in

library management during active work. The cleanup of the manifest layer (MANIFEST.md and PROJECTS.md) ensures the "Ops Lane" remains precise and free of "underscore soup."

Works cited

1. dump-full-main-e29ab6b23.txt